

Laboratorio 3

Deep Learning: Reconocimiento de Lenguaje de Señas (ASL). Entrega Final

Integrantes:

231143 – Dulce Rebeca Ambrosio Jiménez

231177 – Daniel Alejandro Chet Delgado

231135 – Javier Alexander Linares Chang

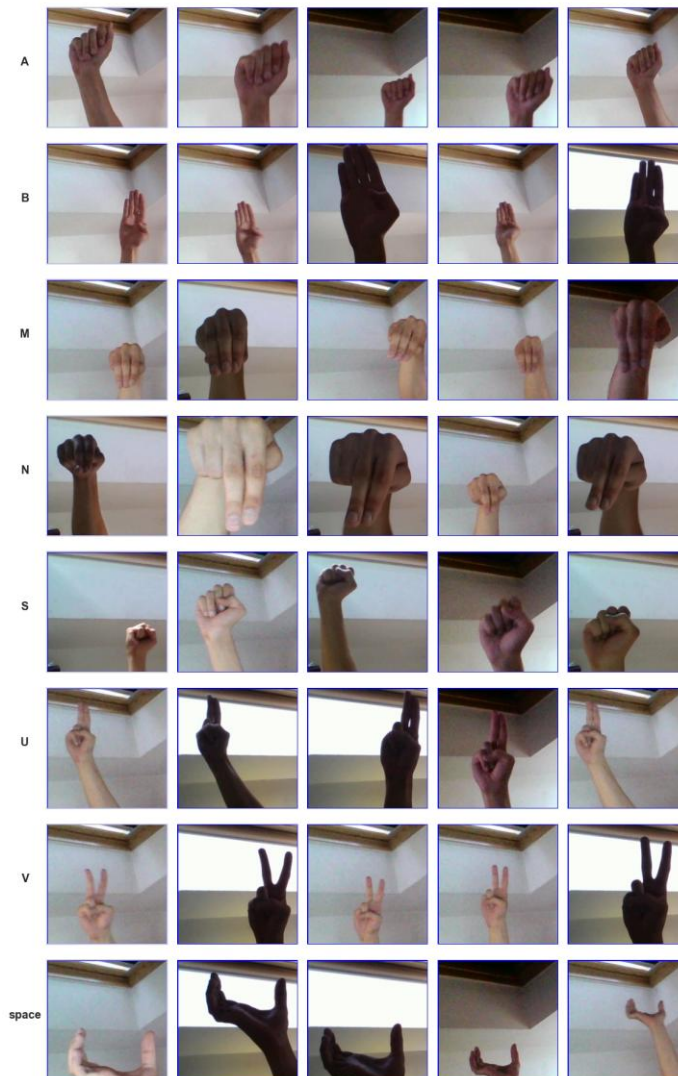
El presente laboratorio corresponde al prototipo del motor de reconocimiento de letras del alfabeto de Lenguaje de Señas Americano (ASL) para SignBridge, una startup de tecnología inclusiva que desarrolla un traductor de señas en tiempo real. El dataset utilizado (ASL Alphabet, Kaggle) es de uso exclusivamente académico.

1. Análisis exploratorio y preprocesamiento (ejercicios 1-3)

El dataset ASL Alphabet trae 3,000 imágenes por clase (87,000 en total), de 200x200 px a color, organizadas en 29 carpetas (A-Z, SPACE, DELETE y NOTHING). Por el costo de cómputo que implica entrenar varios modelos sobre ese volumen en el tiempo de un laboratorio, se trabaja con una submuestra aleatoria de 600 imágenes por clase (17,400 en total), extraída del .zip descargado de Kaggle. El muestreo se hizo de forma aleatoria y no tomando los primeros N archivos de cada carpeta, porque esos archivos corresponden a frames consecutivos de un mismo video y habrían introducido menos variabilidad real dentro de cada clase.

1.1 Ejemplos de letras y variabilidad dentro de una clase. Se muestran 5 muestras de 8 letras distintas (A, B, M, N, S, U, V y la clase SPACE) para observar tanto la forma de cada seña como la variabilidad dentro de una misma clase.

Ejercicio 1 — Variabilidad dentro de cada clase (5 muestras por letra)



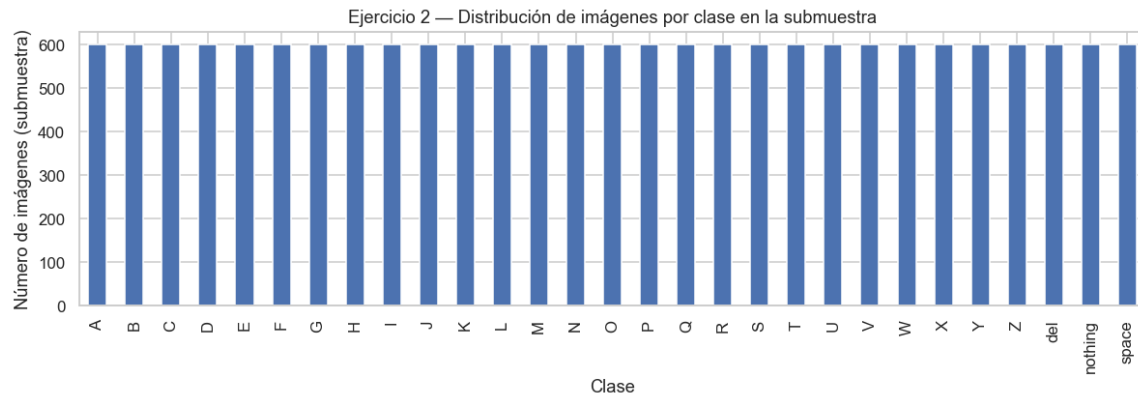
Ejercicio 1 - variabilidad dentro de 8 clases (5 muestras por letra).

Dentro de una misma clase, la mano ocupa casi todo el encuadre sobre un fondo relativamente constante (misma habitación y superficie), y la variabilidad observada viene sobre todo de rotación leve de la muñeca, iluminación y microdiferencias en la pose. Esto es consistente con que las imágenes de una misma clase provienen de pocos videos grabados en sesiones cortas, más que de fotos de muchas personas distintas.

1.2 Resolución y formato de las imágenes. Sobre una muestra de 30 archivos por clase se confirma que todas las imágenes son JPG de 200x200 px en color (RGB), tal como indica el enunciado. El peso de archivo varía de forma moderada (entre 10 y 16 KB aproximadamente), lo que sugiere fondos y niveles de detalle similares entre clases, sin outliers evidentes de resolución o formato.

1.3 Distribución de clases: ¿está balanceado el dataset? El dataset original, y por lo tanto la submuestra tomada con el mismo número fijo por clase, está perfectamente balanceado: 3,000

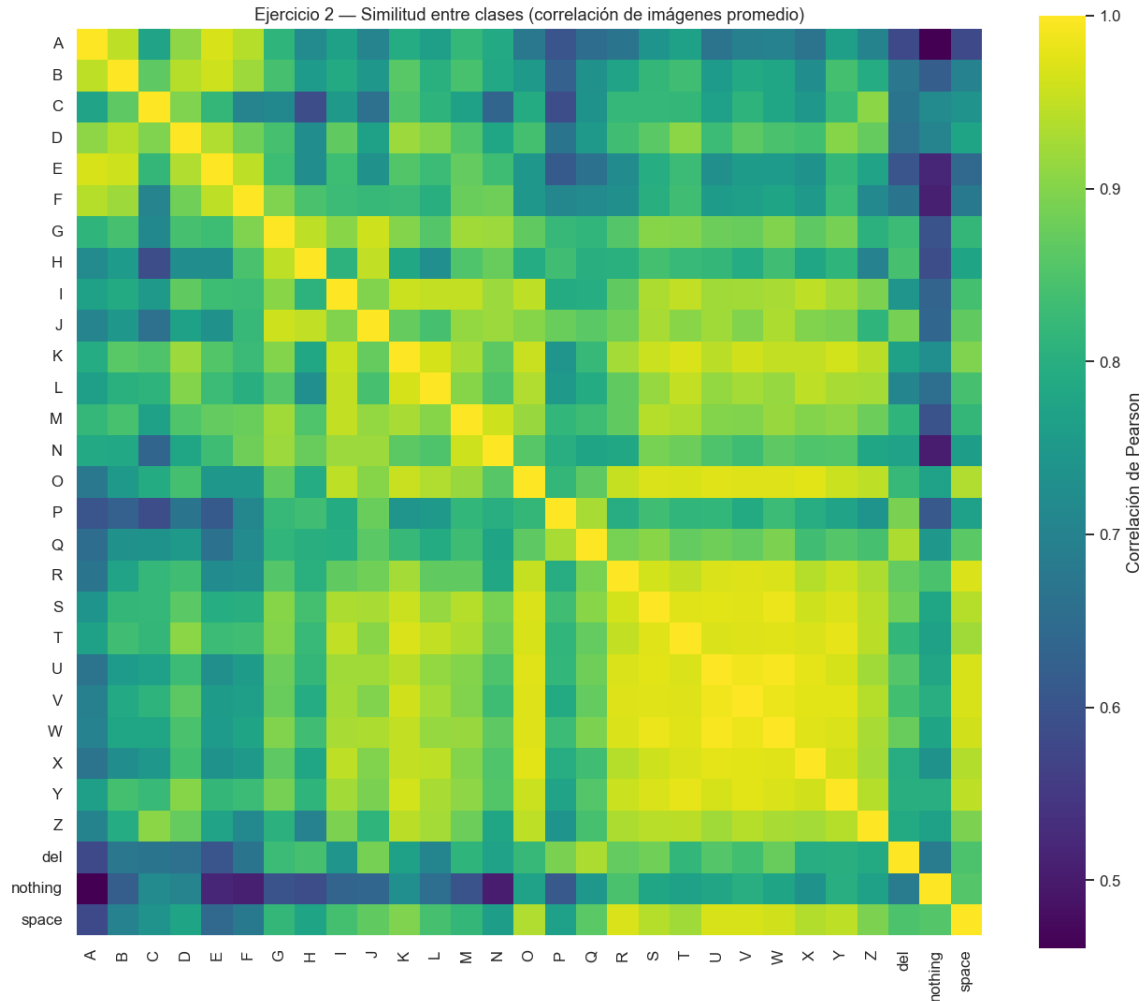
imágenes por clase en el set completo de Kaggle y 600 por clase en la submuestra usada en este laboratorio.



Ejercicio 2 - distribución de imágenes por clase en la submuestra (600 img/clase, 29 clases).

Al no haber clases minoritarias no es necesario ponderar la función de pérdida ni aplicar sobremuestreo o submuestreo antes de entrenar los modelos.

1.4 ¿Qué letras se confunden visualmente entre sí? Para evidenciar esto de forma más objetiva que a simple vista, se calcula la imagen promedio por clase (en escala de grises, redimensionada a 64x64) y se compara cada par de clases con la correlación de Pearson entre esos promedios. Letras con formas de mano parecidas deberían tener imágenes promedio muy correlacionadas.



Ejercicio 2 - correlación de Pearson entre las imágenes promedio de las 29 clases.

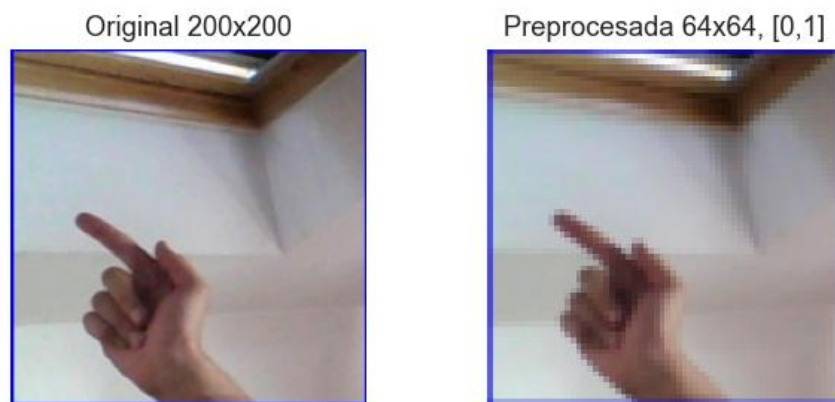
El par más correlacionado de toda la matriz es U-W (0.994), y el top 15 global queda dominado por un grupo más amplio de letras con los dedos índice, medio y anular extendidos hacia arriba (S, T, U, V, W, X, Y), todas por encima de 0.97 de correlación entre sí. Los pares que el enunciado menciona como ejemplo también se confirman, aunque con una magnitud algo menor: M-N=0.958, M-S=0.941 y N-S=0.889 para el grupo M/N/S, y U-V=0.987, U-R=0.969 y V-R=0.972 para el grupo U/V/R. Las clases NOTHING y, en menor medida, DELETE quedan claramente separadas del resto (correlación baja con casi todas las demás), porque no corresponden a una forma de mano específica sino a la ausencia de seña o a un gesto muy distinto.

1.5 Definición del conjunto de entrenamiento, validación y prueba. El set de prueba oficial de Kaggle trae solo una imagen por clase (29 en total), intencionalmente insuficiente para evaluar un modelo con rigor estadístico. Por eso se construye un split propio a partir de la submuestra de 17,400 imágenes: 70% entrenamiento, 15% validación y 15% prueba (12,180 / 2,610 / 2,610 imágenes), estratificado por clase y con semilla fija (42) para que sea reproducible. El set de

prueba oficial de Kaggle se reserva aparte como una prueba adicional fuera de distribución, ya que corresponde a fotos con fondo y persona distintos a los del set de entrenamiento.

1.6 Preprocesamiento de las imágenes. Antes de entrenar se aplican cuatro transformaciones a cada imagen. Redimensión de 200x200 a 64x64 px, que reduce el costo computacional cerca de 10 veces sin perder la información relevante, ya que la mano ocupa la mayor parte del encuadre en casi todas las imágenes. Normalización de los píxeles al rango $[0,1]$ dividiendo entre 255, estándar para facilitar la convergencia del optimizador. Conservación del color (RGB) en vez de convertir a escala de grises, porque el tono de piel y las sombras entre dedos son señales que una CNN puede aprovechar para separar clases visualmente similares como M/N/S o U/V/R. Y finalmente, no se aplican filtros de suavizado ni de ruido, porque las fotos ya son nítidas y con fondo limpio, así que un filtro adicional solo arriesgaría borrar detalle útil sin corregir ningún problema real de los datos.

Ejercicio 3 — Ejemplo de preprocesamiento (clase G)



Ejercicio 3 - ejemplo de preprocesamiento (clase G): original 200x200 vs. preprocesada 64x64 en $[0,1]$.

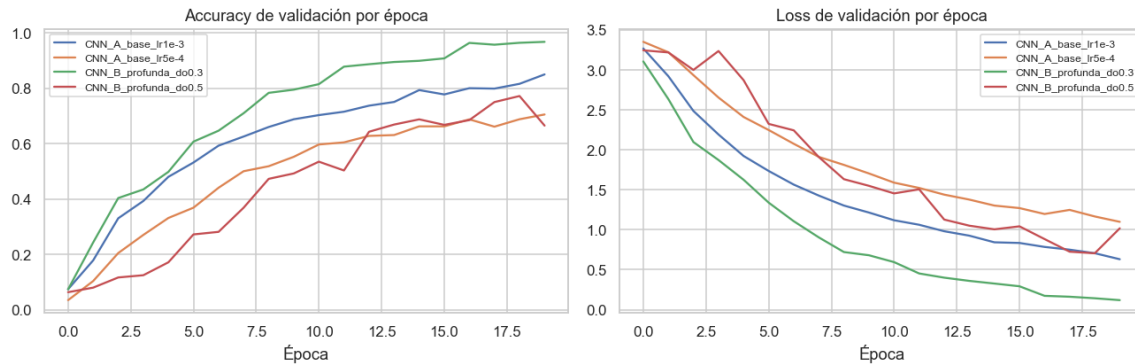
2. Modelos de Deep Learning para imágenes (ejercicios 4-6)

Todos los modelos se entrenan sobre el mismo conjunto de entrenamiento, se seleccionan (hiperparámetros y early stopping) con el conjunto de validación, y se reportan al final sobre el conjunto de prueba, que nunca se usa durante el ajuste. Se reporta accuracy y F1 macro, que pondera igual a cada una de las 29 clases y es relevante dado el grupo de letras confundibles identificado en el análisis exploratorio.

2.1 Redes neuronales convolucionales (CNN). Se entrenan dos arquitecturas, cada una con dos variantes de hiperparámetros (cuatro corridas en total). La arquitectura A (base) apila tres bloques de convolución y max pooling (32, 64 y 128 filtros), seguidos de global average pooling, dropout y una capa densa softmax de 29 salidas. La arquitectura B (profunda) agrega BatchNormalization después de cada convolución, dos convoluciones por bloque en vez de una, y una capa densa de 256 unidades con dropout adicional antes de la salida.

Config	Arquitectura	Dropout	LR
--------	--------------	---------	----

CNN_A_base_lr1e-3	A base (3 bloques conv+pool)	0.3	1e-3
CNN_A_base_lr5e-4	A base (3 bloques conv+pool)	0.3	5e-4
CNN_B_profunda_do0.3	B profunda (BatchNorm + densa 256)	0.3	2e-4
CNN_B_profunda_do0.5	B profunda (BatchNorm + densa 256)	0.5	2e-4



Ejercicio 4 - accuracy y loss de validación por época para las 4 corridas de CNN.

Gana la configuración CNN_B_profunda_do0.3, con 96.8% de accuracy de validación (96.6% en prueba), por delante de las otras tres corridas. La primera vez que se corrió este notebook, varias configuraciones de la arquitectura profunda colapsaron a un accuracy de validación cercano al azar (~3.4%, equivalente a 1 de 29 clases): la tasa de aprendizaje de 1e-3 combinada con BatchNormalization hacía que el accuracy de entrenamiento subiera con normalidad mientras las estadísticas de validación nunca se estabilizaban y el val_loss se disparaba. Se corrigió bajando el learning rate a 2e-4, usando un momentum de 0.9 en vez del 0.99 por defecto en BatchNormalization, y agregando clipnorm al optimizador Adam.

2.2 Red neuronal simple (fully-connected). Se entrenan dos variantes de una red totalmente conectada sobre los mismos píxeles de entrada ($64 \times 64 \times 3 = 12,288$ valores), sin ninguna capa convolucional: MLP_A con dos capas densas (512 y 256 unidades) y MLP_B con tres capas (1024, 512 y 128 unidades). Ambas aplican BatchNormalization justo después de aplanar la imagen. Esa normalización fue necesaria porque, en la primera corrida, alimentar los 12,288 píxeles crudos directamente a una capa densa de 512 a 1024 unidades con Adam y lr=1e-3 producía un primer paso de actualización tan grande que la mayoría de las neuronas ReLU quedaban muertas desde la primera época (loss estancado en $\ln(29) \approx 3.37$). Normalizar la entrada y bajar el learning rate a 1e-4/5e-5 evitó ese colapso.

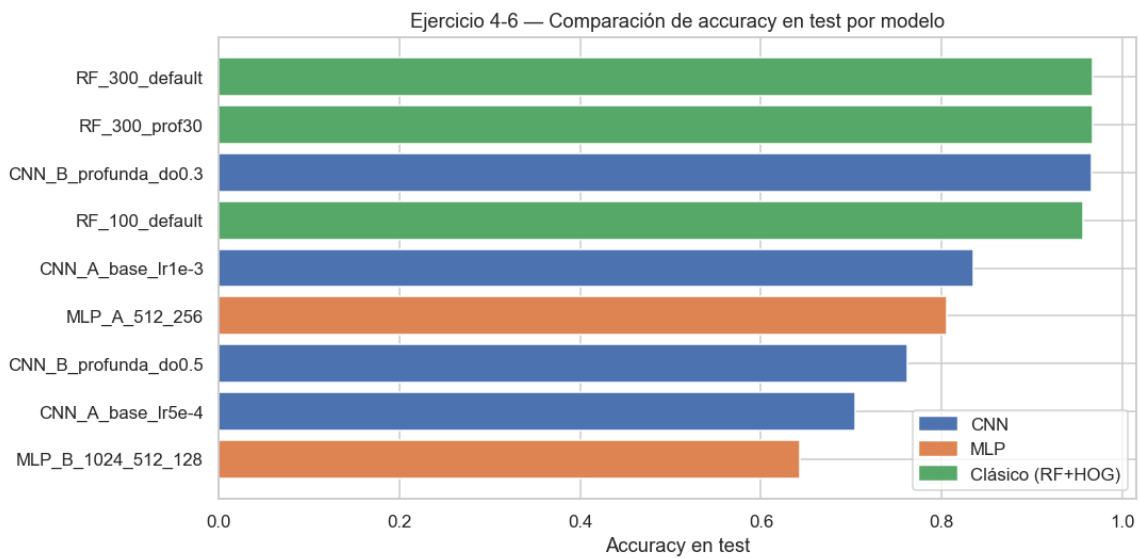
2.3 Modelo con otro algoritmo: Random Forest sobre características HOG. Para un clasificador que no sea de deep learning, los tres candidatos naturales sobre imágenes de manos son Random Forest, SVM y KNN. Se elige Random Forest porque no requiere escalar las características ni asumir un kernel o una métrica de distancia, es más robusto a la alta dimensionalidad de las características HOG, es rápido de entrenar y de evaluar en CPU sobre unas 12,000 muestras (a diferencia de KNN, lento en inferencia, mala señal para un producto de reconocimiento en tiempo real, o de SVM con kernel no lineal, que escala mal con el número de muestras), y además da una medida de importancia de variables. En vez de píxeles crudos, donde

un Random Forest no captura bien relaciones espaciales, se usan características HOG (Histogram of Oriented Gradients), que resumen bordes y orientación de contornos de la mano. Se entrenan tres configuraciones variando el número de árboles y la profundidad máxima.

Config	n_estimators	max_depth
RF_100_default	100	None
RF_300_prof30	300	30
RF_300_default	300	None

2.4 Comparación de los modelos base. La tabla siguiente resume los 9 modelos entrenados hasta este punto (sin augmentation), ordenados de mayor a menor accuracy de validación.

Modelo	Categoría	Val acc	Test acc	Test F1 macro
CNN_B_profunda_do0.3	CNN	96.8%	96.6%	96.6%
RF_300_default	Clásico (RF+HOG)	96.7%	96.7%	96.7%
RF_300_prof30	Clásico (RF+HOG)	96.4%	96.7%	96.7%
RF_100_default	Clásico (RF+HOG)	95.7%	95.6%	95.6%
CNN_A_base_lr1e-3	CNN	85.0%	83.4%	83.5%
MLP_A_512_256	MLP	81.9%	80.5%	81.9%
CNN_B_profunda_do0.5	CNN	77.2%	76.2%	76.3%
CNN_A_base_lr5e-4	CNN	70.5%	70.4%	70.0%
MLP_B_1024_512_128	MLP	66.4%	64.3%	63.9%



Comparación de accuracy en el conjunto de prueba para los 9 modelos base.

El resultado más llamativo es que Random Forest sobre HOG (RF_300_default, 96.7% de accuracy en prueba) prácticamente empata con la mejor CNN (CNN_B_profunda_do0.3, 96.6%), y ambos superan claramente a la mejor MLP (MLP_A_512_256, 80.5%). Esto es consistente con lo esperado: las convoluciones (y, en este caso, también las características HOG hechas a mano) explotan la estructura espacial 2D de la imagen, es decir, bordes, texturas locales de dedos y contornos de la mano, con muchos menos parámetros efectivos que una capa densa conectada directamente a los 12,288 píxeles de entrada. La red fully-connected tiene que aprender esas relaciones espaciales desde cero para cada posición de la imagen, lo cual es más difícil de optimizar y más propenso a sobreajuste con las cerca de 12,000 imágenes de entrenamiento disponibles.

3. Image augmentation (ejercicio 7)

3.1 ¿Por qué un flip horizontal podría cambiar el significado de una seña? El alfabeto ASL usa formas de mano donde la posición relativa de los dedos y el pulgar es la información que distingue una letra de otra (por ejemplo, M/N/S se diferencian por dónde queda el pulgar respecto a los demás dedos, y R se forma cruzando el índice sobre el medio en una dirección específica). Al reflejar horizontalmente la imagen, esa relación espacial se invierte: lo que era el pulgar a la derecha, debajo del índice, pasa a verse como el pulgar a la izquierda, debajo del meñique. Para señas donde esa asimetría es la seña misma, el flip no genera una versión aumentada de la misma letra, sino una imagen que ya no corresponde a esa letra: en el peor caso corresponde a otra letra real, o a una forma de mano que ningún hablante de ASL produciría. Además, J y Z son señas dinámicas que trazan una trayectoria en el aire, y un flip horizontal invertiría la dirección de ese trazo. Por eso el flip horizontal arriesga cambiar la etiqueta real de la imagen sin cambiar la etiqueta que se le asignó, el peor tipo de ruido posible para un dataset supervisado.

3.2 ¿Qué transformaciones sí tienen sentido para señas y cuáles no? Sí tienen sentido las perturbaciones que un mismo firmante produciría de forma natural entre una repetición y otra de la misma seña sin alterar su significado: rotaciones pequeñas de ± 10 -15 grados por la variación natural del ángulo de la muñeca o la cámara, zoom o escala leve por la distancia variable de la mano a la cámara, traslación leve porque la mano no siempre está perfectamente centrada, y variación de brillo y contraste por diferencias de iluminación entre videos o sesiones. No tienen sentido el flip horizontal ni el vertical, por las razones ya explicadas; rotaciones grandes (mayores a 30-45 grados) o distorsiones fuertes de perspectiva, porque ningún firmante presenta la mano así frente a la cámara y solo agregarían ruido irreal; y cambios de color o tono de piel muy agresivos, que podrían introducir sesgo adicional en vez de reducirlo.

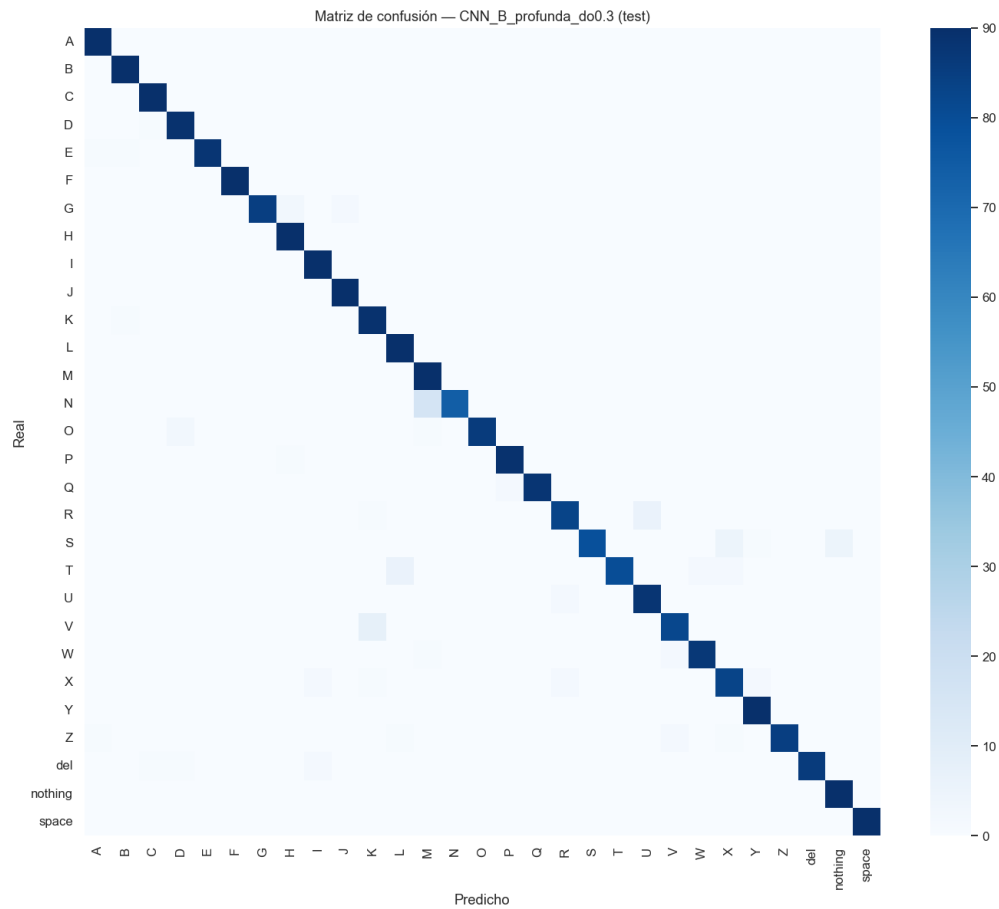
3.3 Resultados con augmentation. Se reentrena la mejor CNN (CNN_B_profunda_do0.3) y la mejor MLP (MLP_A_512_256) agregando rotación, zoom, traslación, brillo y contraste aleatorios, sin flip. El resultado no mejora.

Modelo	Sin augmentation (test acc)	Con augmentation (test acc)
CNN_B_profunda_do0.3	96.6%	90.2%
MLP_A_512_256	80.5%	50.2%

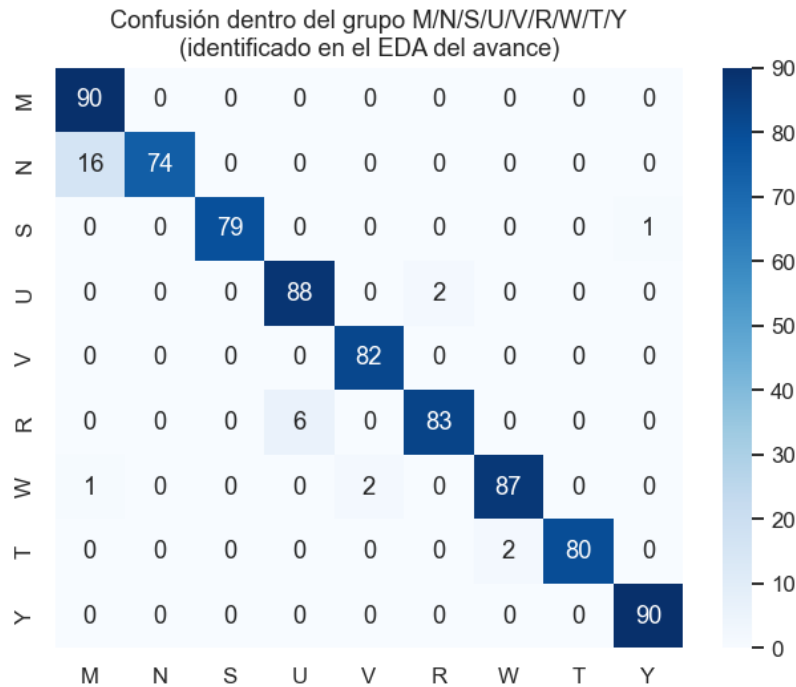
Esto es consistente con que ambos modelos base ya generalizaban razonablemente bien dentro de la distribución del dataset, con poca diferencia entre accuracy de entrenamiento y de validación, así que entrenar sobre versiones perturbadas de las imágenes es una tarea más difícil que no aporta con el mismo número de épocas de entrenamiento. El efecto esperado sería distinto, y probablemente más beneficioso, sobre un dataset con menos ejemplos por clase o con mayor variabilidad real de firmantes y cámaras, que es justo lo que le falta a este dataset y se discute en la reflexión de accesibilidad.

4. Selección del mejor modelo y prueba con señas propias (ejercicio 8)

4.1 Selección del mejor modelo global. Considerando los 11 modelos entrenados (9 base más 2 con augmentation), el mejor modelo por accuracy de validación es CNN_B_profunda_do0.3, con 96.6% de accuracy y 96.6% de F1 macro sobre el conjunto de prueba, prácticamente empatado con RF_300_default (96.7%/96.7%). Se guarda como el modelo global del laboratorio (mejor_modelo.keras).



Matriz de confusión de CNN_B_profunda_do0.3 sobre las 29 clases del conjunto de prueba.



Matriz de confusión restringida al grupo de letras visualmente similares identificado en el análisis exploratorio (M, N, S, U, V, R, W, T, Y).

Los errores del modelo se concentran casi por completo en ese grupo de letras: la confusión más marcada es N predicha como M (16 de 90 casos), seguida de R predicha como U (6 de 90) y errores aislados de 1 o 2 casos entre W/V y T/R. Fuera de ese grupo la diagonal es prácticamente perfecta, lo que confirma que el modelo separa bien la gran mayoría del alfabeto y que sus pocos errores caen exactamente donde el análisis exploratorio anticipaba mayor dificultad.

4.2 Prueba con fotos propias del grupo. Para evaluar qué tan bien generaliza el mejor modelo fuera del dataset de Kaggle, cada integrante del grupo tomó fotos de 5 letras distintas con su propio celular, en su propia casa: Daniel (K, L, M, N, O), Dulce (A, B, C, D, E) y Javier (F, G, H, I, U), 15 fotos en total.



Predicciones de CNN_B_profunda_do0.3 sobre las 15 fotos propias del grupo (verde = acierto, rojo = error).

El resultado es de apenas 3 aciertos de 15 (20%): 1 de 5 para Daniel, 2 de 5 para Dulce y 0 de 5 para Javier, muy por debajo del 96.6% obtenido sobre el test set del propio dataset. Cuatro de las doce fotos mal clasificadas se predijeron como N (las K, M y O de Daniel, y las G y H de Javier), casi siempre con una confianza superior al 90%, lo que muestra que el modelo no solo se equivoca sino que lo hace con mucha seguridad: el softmax no funciona como una señal confiable de "no sé" ante entradas que se alejan de la distribución de entrenamiento. Los tres aciertos (B, E y N) corresponden a formas de mano muy distintivas, con los dedos muy extendidos o muy cerrados, que sobreviven mejor al cambio de cámara y de fondo que señas con diferencias sutiles en la posición del pulgar. Esta caída de cerca de 77 puntos porcentuales es la evidencia empírica más directa de que el 96.6% de accuracy reportado sobre el dataset mide qué tan bien el modelo interpola dentro de esa distribución particular (una persona, una cámara, un fondo fijo), y no qué tan bien reconocería señas de un usuario real de SignBridge.

5. Reflexión de accesibilidad y sesgo (ejercicio 9)

El dataset ASL Alphabet tiene varias limitaciones relevantes para un producto real de accesibilidad como el que busca SignBridge. En cuanto a tono de piel, el dataset proviene de un número muy reducido de firmantes, aparentemente uno o pocos, con el mismo tono de piel y la misma mano en las cerca de 87,000 imágenes; un modelo entrenado así puede aprender a asociar implícitamente esa mano, ese fondo y esa iluminación con la clase correcta, y fallar sistemáticamente con usuarios de tonos de piel distintos, justo el tipo de sesgo que un producto de accesibilidad no puede permitirse. El resultado del ejercicio 8, con apenas 20% de accuracy sobre

manos y cámaras distintas a las del dataset, es evidencia directa de este riesgo. En cuanto a ángulo de cámara e iluminación, todas las fotos parecen tomadas con la misma cámara fija y la misma iluminación de interior, con fondo limpio; en el uso real de una app en un celular, con el usuario sosteniendo el teléfono con la otra mano, luz variable y fondos con ruido visual, la distribución de entrada será muy distinta a la de entrenamiento. En cuanto a tamaño y forma de la mano, una sola persona no representa la variabilidad real de manos de niños, adultos mayores o personas con diferencias físicas en la mano, como amputaciones parciales o artritis, que son justo parte de la población que un traductor de señas debería servir. Y en cuanto a letras dinámicas simplificadas a poses estáticas, J y Z son señas con movimiento en ASL real, y el dataset las reduce a una sola foto de una pose intermedia, así que un clasificador de imagen fija no puede aprender esa seña de verdad.

Para acercar este prototipo a un caso de uso real como el de SignBridge, la recomendación más importante es recolectar un dataset propio y diverso, con múltiples firmantes de distintos tonos de piel, edades y tipos de mano, grabado con dispositivos y ángulos representativos del uso real, idealmente en colaboración con la comunidad sorda y con organizaciones de personas con discapacidad. Como paso intermedio de menor costo, se podría anteponer un detector o normalizador de mano, por ejemplo con los landmarks de MediaPipe Hands, antes del clasificador, para que el modelo trabaje sobre la geometría de la mano en vez de sobre los píxeles crudos y así reducir la dependencia del tono de piel, el fondo y la iluminación. Para J y Z, y para robustez general, valdría la pena pasar de clasificar imágenes estáticas a clasificar secuencias cortas de video. Y antes de lanzar el producto, convendría medir el desempeño desagregado por subgrupo (tono de piel, edad, dispositivo) y no solo el accuracy global, para detectar sesgo que un promedio agregado favorable podría estar escondiendo.

Conclusiones generales

Entre los tres bloques de este laboratorio, la conclusión que más se repite es que más complejidad no es lo mismo que mejor desempeño, aunque con matices distintos en cada caso. Entre los modelos base, la mejor CNN (96.6% de accuracy en prueba) prácticamente empata con un Random Forest sobre características HOG (96.7%), un modelo mucho más simple y rápido de entrenar, mientras que la red fully-connected, con la misma cantidad de píxeles de entrada y muchos más parámetros que cualquiera de los dos, queda casi 16 puntos por debajo (80.5%): para este dataset relativamente limpio, de una sola persona y una sola cámara, tanto la convolución como las características hechas a mano capturan casi toda la señal relevante, y lo que realmente penaliza el desempeño es no explotar en absoluto la estructura espacial de la imagen. El image augmentation, aplicado deliberadamente sin flip horizontal ni vertical para no invertir el significado de las señas, tampoco mejoró los resultados sobre este dataset ya bien ajustado (la CNN bajó a 90.2% y la MLP a 50.2%), aunque la discusión de qué transformaciones tienen sentido para señas seguiría siendo relevante sobre un dataset con más variabilidad real de firmantes. El resultado más importante para SignBridge como producto es, sin embargo, la prueba con fotos propias del grupo: apenas 20% de accuracy (3 de 15) con el mejor modelo, muy por debajo del 96.6% reportado sobre el test set del propio dataset. Esa caída de cerca de 77 puntos porcentuales muestra en la práctica lo que la reflexión de accesibilidad anticipa en teoría: un dataset construido con una sola persona, una sola cámara y una sola condición de iluminación

puede rendir muy bien dentro de su propia distribución y, al mismo tiempo, generalizar muy mal a usuarios reales, que es exactamente el riesgo que un producto de accesibilidad no puede permitirse sin antes recolectar datos más diversos y medir su desempeño desagregado por subgrupo.